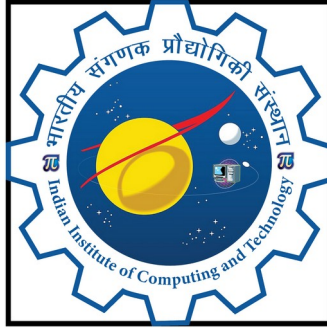




भारतीय संगणक एवं प्रौद्योगिकी संस्थान
INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY
AFFILIATED: I-STEM, OFFICE OF THE PRINCIPAL SCIENTIFIC ADVISER TO THE GOVERNMENT OF INDIA

PhishGuard AI: A Machine Learning Approach to Phishing Email Detection



Project Report

15th June to 30th July 2026

Student Name: Shreya G Amin

Institute Name: NMAM Institute of Technology, Nitte

Institute Roll no.: NNM24CS352

Enrollment no.: 760225

भारतीय संगणक एवं प्रौद्योगिकी संस्थान | INDIAN INSTITUTE OF COMPUTING AND TECHNOLOGY

Add.: Veer Savarkar Block Shakarpur New Delhi - 1100092

Ph.: 7290006670 | Email: contact@iict.edu.in | Website: www.iict.edu.in

PhishGuard AI: A Machine Learning Approach to Phishing Email Detection

Shreya G Amin

Department of Computer Science and Engineering

NMAM Institute of Technology, Nitte, Karnataka, India

GitHub: github.com/SHREYA-G-AMIN/AI-Driven-Phishing-Email-Detection

Live Demo: ai-driven-phishing-email-detection.streamlit.app

Abstract— Phishing emails remain one of the most common vectors for cyberattacks, deceiving users into revealing sensitive information or credentials through fraudulent messages. This project presents PhishGuard AI, a machine learning system that classifies emails as phishing or legitimate based on their textual content. Emails are preprocessed through lowercasing, punctuation removal, stopword removal, and tokenization, then converted into TF-IDF feature vectors of 5,000 dimensions. Four classification algorithms — Naive Bayes, Logistic Regression, Random Forest, and a Neural Network (Multi-Layer Perceptron) — were trained and evaluated on a dataset of 18,650 emails using accuracy, precision, recall, and F1-score, given a class imbalance of approximately 61% phishing and 39% legitimate emails. The Neural Network achieved the highest accuracy at 96.94%, followed by Logistic Regression at 96.68%. Logistic Regression was selected for production deployment as it offered the best balance between predictive accuracy and inference speed. The final system was deployed as an interactive Streamlit web application that classifies emails in real time, displaying a confidence score, a threat level (High/Medium/Low), and actionable security recommendations to the user.

Index Terms— Phishing Detection, Natural Language Processing, TF-IDF, Machine Learning, Logistic Regression, Email Security, Streamlit

I. INTRODUCTION

Phishing attacks use deceptive emails that impersonate trusted entities to trick recipients into clicking malicious links, downloading harmful attachments, or disclosing confidential information such as passwords and financial details. Despite growing awareness, phishing continues to succeed at scale because attackers constantly vary their language and tactics, making manual identification unreliable. Automated, content-based detection systems that can flag suspicious emails in real time offer a practical layer of defense to supplement traditional spam filters.

Phishing remains one of the most reported categories of cybercrime worldwide, and email continues to be the primary delivery channel because it is inexpensive to send at scale and can be highly convincing when attackers impersonate familiar organizations. This motivates lightweight, content-based classifiers that can be deployed directly at the point

where a user reads an email, rather than relying solely on perimeter-level spam filtering.

This project develops PhishGuard AI, a complete pipeline that takes raw email text as input and predicts whether the email is phishing or legitimate, along with a confidence score and threat level. Multiple machine learning models were trained and compared, and the best-performing model suited to production constraints was deployed through a Streamlit web application with an accompanying dashboard interface.

II. PROBLEM STATEMENT

Given the raw text of an email, the goal is to build a binary classifier that predicts whether the email is “Phishing” or “Legitimate” with high accuracy, while remaining fast enough for real-time use and robust to the class imbalance present in typical phishing datasets, where phishing samples often outnumber legitimate ones. The system must rely only on the email's textual content, without requiring access to sender infrastructure, network-level metadata, or organization-specific email history.

III. LITERATURE SURVEY

Phishing email detection has been studied extensively as an NLP-driven text classification problem. Salloum et al. conducted a literature survey of NLP-based phishing email detection techniques, reviewing feature extraction methods such as bag-of-words and TF-IDF alongside classifiers ranging from Naive Bayes and Support Vector Machines to more recent deep learning approaches, and found that classical machine learning models paired with strong textual features remain widely used due to their accuracy, speed, and lower resource requirements compared to deep learning pipelines [5].

Existing phishing detection systems generally fall into three categories: content-based methods that analyze the text of the email body and subject line, as used in this project; URL and header-based methods that inspect links, sender domains, and header metadata for spoofing indicators; and hybrid methods that combine both textual and structural signals. Content-based approaches remain attractive for lightweight, client-side or browser-integrated tools because they do not require

access to email server infrastructure or header metadata that may be stripped or unavailable in some deployment contexts.

In terms of feature representation, TF-IDF weighting was chosen over dense word embeddings for this project because it produces sparse, interpretable features that pair well with classical classifiers such as Logistic Regression and Random Forest, without the overhead of loading a pre-trained embedding model, keeping inference fast enough for real-time use in a browser-based dashboard.

IV. SYSTEM ARCHITECTURE

The system follows a linear pipeline in which raw email text is progressively transformed into a prediction. Figure 1 illustrates the high-level architecture, from raw input text through preprocessing and feature extraction to model inference and dashboard-based delivery.

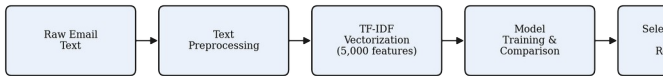


Fig. 1. System architecture of the phishing email detection pipeline.

Each stage is modular: the preprocessing and vectorization steps are shared across all four candidate models, allowing them to be trained and compared fairly on identical input representations before the model best suited to production — Logistic Regression — is frozen and exported for deployment.

V. METHODOLOGY

Component	Technology / Library
Programming Language	Python
Data Handling	Pandas, NumPy
NLP Preprocessing	NLTK
Feature Extraction	TF-IDF (Scikit-learn)
Model Training	Scikit-learn
Model Serialization	Joblib
Web Interface	Streamlit

TABLE II. TECHNOLOGIES AND TOOLS USED

A. Dataset

The model was trained on the “Phishing Email Detection” dataset from Kaggle, contributed by user subhajournal [1]. The dataset contains 18,650 emails in total, comprising 11,322 phishing emails (60.7%) and 7,328 legitimate emails (39.3%). Due to this class imbalance, F1-score was tracked alongside accuracy to ensure the models did not simply favor the majority class.

B. Text Preprocessing

Each email’s text content was cleaned through lowercasing, removal of punctuation and special characters, tokenization, and removal of common English stopwords using NLTK [2], producing a normalized representation suitable for vectorization.

C. Feature Extraction

Cleaned email text was transformed into TF-IDF feature vectors restricted to the top 5,000 terms, capturing the relative importance of words across the email corpus while keeping the feature space computationally manageable.

D. Model Training and Evaluation

Four models were trained on the TF-IDF features using Scikit-learn [3]: Multinomial Naive Bayes, Logistic Regression, Random Forest (50 estimators), and a Neural Network (MLP with hidden layers of 100 and 50 units). Each model was evaluated using accuracy, precision, recall, F1-score, and confusion matrix analysis to account for the class imbalance in the dataset.

Multinomial Naive Bayes estimates the probability of an email being phishing based on word-frequency statistics under an assumption of feature independence, making it extremely fast to train even though the independence assumption rarely holds exactly for natural language. Logistic Regression models this probability as a linear combination of TF-IDF feature weights, offering a fast and interpretable baseline well suited to real-time inference. Random Forest builds an ensemble of decision trees on bootstrapped samples of the training data, which tends to reduce overfitting and captures non-linear interactions between terms at the cost of a larger, slower model. The Neural Network (MLP) learns non-linear transformations of the input features through its hidden layers, allowing it to capture subtler patterns at the cost of requiring more careful tuning and longer inference time.

E. Deployment

Although the Neural Network achieved marginally higher accuracy, Logistic Regression was selected for production deployment as it offered comparable accuracy with significantly faster inference, which is important for a real-time detection tool. The selected model and TF-IDF vectorizer were serialized using Joblib and integrated into a Streamlit dashboard that classifies pasted email text, displaying a phishing/legitimate verdict, a confidence score, a threat level (High/Medium/Low), and corresponding security recommendations.

F. Evaluation Strategy

All four models were trained and evaluated on an identical train-test split so that comparisons between them would be fair. Because the dataset is imbalanced toward phishing emails, precision, recall, and F1-score were tracked in addition to accuracy, and confusion matrix analysis was used to check that legitimate emails were not being disproportionately misclassified as phishing, which would harm usability if deployed.

VI. RESULTS AND DISCUSSION

Rank	Model	Accuracy (%)
1	Neural Network	96.94
2	Logistic Regression	96.68
3	Random Forest	95.76
4	Naive Bayes (MultinomialNB)	95.52

TABLE I. MODEL ACCURACY COMPARISON

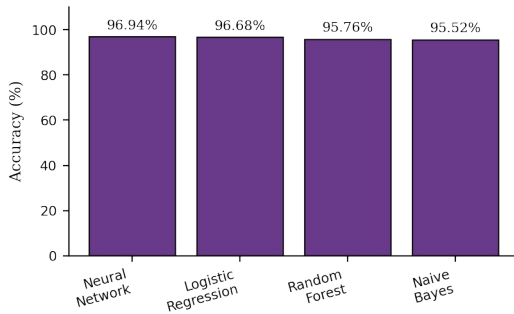


Fig. 2. Test accuracy comparison across the four models.

The Neural Network achieved the highest accuracy at 96.94%, with Logistic Regression close behind at 96.68%, followed by Random Forest at 95.76% and Naive Bayes at 95.52%. The narrow spread between all four models indicates that the TF-IDF features were highly discriminative for this task regardless of model complexity. Given the small accuracy gap between the top two models, Logistic Regression's lower computational overhead made it the more practical choice for real-time deployment, illustrating that the highest-accuracy model is not always the best production choice when inference speed and simplicity also matter.

Because the dataset is moderately imbalanced (60.7% phishing vs. 39.3% legitimate), precision, recall, and F1-score were considered alongside raw accuracy during evaluation to confirm that models were not simply biased toward predicting the majority phishing class.

These results align with the broader literature on phishing email detection, which similarly reports that classical machine learning models achieve strong accuracy on TF-IDF or bag-of-words representations of email text, often matching or approaching the performance of more complex deep

learning approaches while remaining considerably cheaper to train and deploy [5].

Model	Strength	Weakness
Naive Bayes	Very fast, simple probabilistic model	Assumes feature independence
Logistic Regression	Fast, interpretable, low latency	Assumes linear decision boundary
Random Forest	Robust, handles non-linearity well	Larger model size, slower inference
Neural Network	Captures complex patterns	Needs more tuning, less interpretable

TABLE III. QUALITATIVE COMPARISON OF MODELS

Table III summarizes the qualitative trade-offs behind the accuracy figures in Table I. Naive Bayes performed only slightly below the other models despite its strong independence assumption, suggesting that individual discriminative words (such as urgency phrases or credential-related terms) carry most of the predictive signal in this dataset. Logistic Regression and Random Forest both benefited from the near-linear separability of TF-IDF features for this task, while the Neural Network's added flexibility gave it a small additional edge at the cost of longer inference time, motivating the choice of Logistic Regression for the latency-sensitive deployment.

VII. LIMITATIONS

- **Dataset scope:** the model is trained on a specific email dataset and may not generalize equally well to phishing styles, languages, or formats not represented in the training data, such as emerging social-engineering tactics.
- **Class imbalance:** the 60/40 split between phishing and legitimate emails may still bias predictions on borderline legitimate emails despite the use of F1-score during evaluation.
- **Text-only analysis:** the system analyzes only the textual content of an email and does not inspect attachments, embedded images, or sender header metadata, which are also common phishing indicators.
- **Static training data:** the model does not automatically adapt to new phishing tactics or campaigns without periodic retraining on fresh data.

VIII. ADVANTAGES AND APPLICATIONS

The proposed system offers several practical advantages over heavier, transformer-based alternatives. Because it relies on TF-IDF features and a linear classifier, it trains quickly on commodity hardware, produces a small, easily serialized model, and returns predictions in well under a second, making it suitable for real-time, browser-based use without requiring a GPU or cloud inference endpoint.

- Real-time email client plugin that flags suspicious emails before a user clicks a link or downloads an attachment.
- First-pass triage tool for security teams reviewing large volumes of reported or quarantined emails.
- Security awareness training tool that helps employees recognize linguistic patterns common in phishing attempts.
- Lightweight deployment in small organizations or personal email setups where enterprise-grade security suites are impractical.

IX. ETHICAL CONSIDERATIONS

An automated phishing classifier carries real risks if deployed without care. A false positive that flags a legitimate email as phishing could cause a user to miss an important message or lose trust in the tool, while a false negative that misses a genuine phishing email could leave a user exposed to real harm. For this reason, the tool built in this project is intended as a decision-support aid that presents a confidence score and threat level for user judgment, rather than an automated system that silently deletes or blocks emails without any visibility to the user.

There is also a responsibility to keep the training data and resulting model reasonably up to date, since phishing tactics evolve quickly; a model evaluated only once on a static dataset can give a false sense of security if deployed unchanged for a long period without periodic retraining and revalidation against newer phishing samples.

Finally, because the dataset used here skews toward phishing emails (60.7%), care should be taken when reporting results to avoid overstating real-world performance: a classifier can appear highly accurate on an imbalanced test set while still making a meaningful number of errors on the minority legitimate-email class, which is why precision, recall, and F1-score were tracked in this project rather than accuracy alone.

X. CONCLUSION AND FUTURE SCOPE

PhishGuard AI demonstrates that a TF-IDF-based machine learning pipeline can effectively distinguish phishing emails from legitimate ones, with all four evaluated models exceeding 95% accuracy on a dataset of 18,650 emails. Logistic Regression was deployed to production due to its strong balance of accuracy and speed, and the system was made accessible through a real-time Streamlit dashboard with confidence scoring and threat-level classification.

Future work could extend the system in the following directions:

- Analyzing email headers, sender reputation, and embedded URLs alongside body text to catch

phishing attempts that rely more on structural spoofing than language.

- Exploring transformer-based models for deeper contextual understanding of more sophisticated, low-signal phishing attempts, building on directions identified in the wider literature [5].
- Periodically retraining on newly reported phishing samples to keep pace with evolving attacker tactics.
- Adding explainability features that highlight the specific phrases driving a phishing verdict, to help users learn to recognize similar attempts themselves.

APPENDIX

The repository is organized as follows, separating raw data, trained model artifacts, the training notebook, generated reports, screenshots, and the Streamlit application code:

```
AI-Driven-Phishing-Email-Detection/ | data/ |
| phishing_email.csv | models/ |
| lr_model.pkl | tfidf.pkl | notebooks/ |
| phishing_email_detection.ipynb |
screenshots/ | app_homepage.png |
phishing_detected.png | safe_email.png |
reports/ | IEEE_Report.pdf | app.py |
requirements.txt | README.md
```

Running the Project Locally

The project can be reproduced locally by cloning the repository, installing the dependencies listed in requirements.txt, downloading the dataset from Kaggle into the data/ folder, running all cells of the training notebook to regenerate the model and vectorizer artifacts, and finally launching the dashboard with the command: `python -m streamlit run app.py`. This launches the same interface used for the live deployment, allowing predictions to be tested locally before any redeployment.

ACKNOWLEDGMENT

The author would like to thank the mentors and coordinators of the Indian Institute of Computing and Technology (IIT) internship program for their guidance and support throughout the development of this project.

REFERENCES

- [1] Subhajournal, "Phishing Email Detection," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/subhajournal/phishing-email-detection>
- [2] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python, O'Reilly Media, 2009.
- [3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [4] Streamlit Inc., Streamlit Documentation. [Online]. Available: <https://docs.streamlit.io>

[5] S. Salloum, T. Gaber, S. Vadera, and K. Shaalan, "Phishing Email Detection Using Natural Language Processing Techniques: A Literature Survey," *Procedia Computer Science*, vol. 189, pp. 19–28, 2021.